

std_enroll_no	std_roll_no	std_name	address	subject	marks
DS1001	1001	Amit	Indore	DBMS	80
DS1002	1002	Sumit	Bhopal	Python	70
DS1003	1003	vinit	Dhar	DBMS	90
DS1004	1004	Deepak	Ujjan	OR	60
DS1005	1005	Ajay	Indore	DBMS	80
DS1006	1006	Raj	Indore	Python	90

rows in set (0.00 sec)

1. Find all students who scored above 80 marks.sql>

```
Sql> SELECT std_name
FROM student
WHERE marks > 80;
```

2. Count the number of students enrolled in each subject.

```
Sql>SELECT subject, COUNT(*) AS "Student_Count"
FROM student
GROUP BY subject;
```

3. Retrieve the names of students who are studying "DBMS".

```
Sql>SELECT std_name
FROM student
WHERE subject= "DBMS";
```

4. Find the average marks of students in each subject.

```
Sql>SELECT subject, AVG(marks) AS "average_marks"
FROM student
GROUP BY subject;
```

5. List students and their marks, sorted by marks in descending order.

```
Sql>SELECT std_name, marks
FROM student
ORDER BY marks DESC;
```

6. Find the student with the highest marks.

```
Sql>SELECT student, marks
FROM student
ORDER BY marks DESC
LIMIT 1;
```

7. Retrieve students who have marks less than the average marks of the entire class.

```
Sql>SELECT student_name
FROM student
WHERE marks < (SELECT AVG(marks) FROM student_records);
```

8. Find the total marks scored by all students.

```
Sql>SELECT SUM(marks) AS "Total_Marks" FROM student;
```

9. Determine the highest score in each subject.

```
Sql>SELECT subject, MAX(marks) AS "highest_score"  
FROM student  
GROUP BY subject;
```

10. Count the number of students who scored below 50 marks.

```
Sql>SELECT COUNT(*) AS "students_below_50"  
FROM student  
WHERE marks < 50;
```

11. Calculate the average marks for each subject.

```
Sql>SELECT subject, AVG(marks) AS "average_marks"  
FROM student  
GROUP BY subject;
```

12. Find the number of distinct subject offered.

```
Sql>SELECT COUNT(DISTINCT subject) AS "distinct_courses"  
FROM student;
```

13. Get the total number of students enrolled in each subject.

```
Sql>SELECT subject, COUNT(*) AS "student_count"  
FROM student  
GROUP BY subject;
```

14. Retrieve the minimum marks achieved in the entire student table.

```
Sql>SELECT MIN(marks) AS "minimum_marks"  
FROM student;
```

15. Find the total marks for each student.

```
Sql>SELECT std_name, SUM(marks) AS "total_marks"  
FROM student  
GROUP BY std_name;
```

16. Count how many students received marks higher than the average.

```
Sql>SELECT COUNT(*) AS "students_above_average"  
FROM student  
WHERE marks > (SELECT AVG(marks) FROM student);
```

17. Find the subject with the highest average marks.

```
Sql>SELECT subject  
FROM student  
GROUP BY subject  
ORDER BY AVG(marks) DESC  
LIMIT 1;
```

18. Calculate the percentage of marks scored by each student.

```
Sql>SELECT std_name, marks, (marks / 100.0) * 100 AS percentage  
FROM student;
```

19. Round the marks of each student to the nearest whole number.

```
Sql>SELECT std_name,  
ROUND(marks) AS "rounded_marks"  
FROM student;
```

20. Calculate the difference between the highest and lowest marks in the class.

```
Sql>SELECT MAX(marks) - MIN(marks) AS "Marks_Difference"  
FROM student;
```

21. Find students who scored more than 10% above the average marks.

```
Sql>SELECT std_name, marks  
FROM student  
WHERE marks > (SELECT AVG(marks) * 1.10 FROM student);
```

22. Find the marks and square each student's marks.

```
Sql>SELECT std_name, marks, POWER(marks, 2) AS "marks_squared"  
FROM student;
```

23. Calculate the average marks and round it to two decimal places.

```
Sql>SELECT ROUND(AVG(marks), 2) AS "average_marks"  
FROM student;
```

24. Determine the percentage contribution of each student's marks to the total marks.

```
Sql>SELECT std_name, marks, (marks / (SELECT SUM(marks) FROM student) * 100) AS  
"percentage_contribution"  
FROM student;
```

25. Get the absolute difference between each student's marks and the average marks.

```
Sql>SELECT std_name, marks, ABS(marks - (SELECT AVG(marks) FROM student))  
AS "marks_difference"  
FROM student;
```

26. Find the rank of each student based on their marks using a window function (if supported by your SQL dialect).

```
Sql>SELECT std_name, marks, RANK() OVER (ORDER BY marks DESC) AS rank  
FROM student;
```

27. Counts the total number of students in the table.

```
Sql>SELECT COUNT(*) AS "total_students" FROM students;
```

28. Calculates the length of each student's name.

```
Sql>SELECT std_name, LENGTH(std_name) AS "first_name_length" FROM students;
```

29. Converts all last names to uppercase letters.

```
Sql>SELECT UPPER(std_name) AS "uppercase_name" FROM students;
```

30. Converts all first names to lowercase letters.

```
Sql>SELECT LOWER(std_name) AS "lowercase_student_name" FROM students;
```

31. Extracts the first two letters of each student's first name.

```
Sql>SELECT std_name, SUBSTRING(std_name, 1, 2) AS "first_two_letters" FROM students;
```

32. Retrieves all students whose names start with the letter 'S'.

```
Sql>SELECT * FROM student  
WHERE std_name LIKE 'S%';
```

33. Retrieves students whose names have more than four characters.

```
Sql>SELECT * FROM students  
WHERE LENGTH(first_name) > 3;
```

34. Replaces the letter 'o' with '0' in the std_name.

```
Sql>SELECT std_name, REPLACE(std_name, 'o', '0') AS "modified_last_name" FROM students;
```

35. Adds a new column email to the students table to store email addresses.

```
Sql>ALTER TABLE student  
ADD email VARCHAR(100);
```

36. Renames the std_name column to std_first_name

```
Sql>ALTER TABLE students  
RENAME COLUMN std_name TO std_first_name;
```

37. Removes the subject column from the student table.

```
Sql>ALTER TABLE students  
DROP COLUMN subject;
```

38. Sets the student_id column as the primary key for the table.

```
Sql>ALTER TABLE students  
ADD PRIMARY KEY (student_id);
```

39. Ensures that all values in the email column are unique.

```
Sql>ALTER TABLE students  
ADD CONSTRAINT unique_email UNIQUE (email);
```

40. Increases the marks of all students by 5 points.

```
Sql>UPDATE students  
SET marks = marks + 5;
```

41. Changes Amit name to Amit kumar

```
Sql>UPDATE students  
SET std_name = 'Amit Kumar'  
WHERE std_enroll_no = "DS1001";
```

42. Sets a default marks of 80 for any students whose marks is currently null.

```
Sql>UPDATE student  
SET grade = 80  
WHERE marks IS NULL;
```

43. Removes Amit from the students table.

```
Sql>DELETE FROM student  
WHERE std_name ="Amit";
```

44. Deletes all students whose name start from “A”

```
Sql>DELETE FROM student  
WHERE std_name LIKE “A%”;
```

45. Removes all students whose marks is below 75.

```
Sql>DELETE FROM student  
WHERE marks < 75;
```

46. Removes all records from the students table but keeps the table structure intact.

```
Sql>DELETE FROM students;
```

47. Retrieves all columns and records from the students table.

```
Sql>SELECT * FROM students;
```

48. Retrieves only the first_name and last_name columns for all students.

```
Sql>SELECT first_name, last_name FROM students;
```

49. Finds all students with a marks of 90.

```
Sql>SELECT * FROM students  
WHERE marks = 90;
```

50. Retrieves all students whose marks are greater than 80.

```
Sql>SELECT * FROM student  
WHERE marks > 80;
```

51. Retrieves all students with the name Rj.

```
Sql>SELECT * FROM student  
WHERE std_name = “Raj”;
```

52. Retrieves all students sorted by their marks in descending order.

```
Sql>SELECT * FROM students  
ORDER BY marks DESC;
```

53. Counts the total number of students in the table.

```
Sql>SELECT COUNT(*) AS “total_students” FROM student;
```

54. Groups students by their subject and counts how many students are in each subject.

```
Sql>SELECT marks, COUNT(*) AS “Totakl Student”  
FROM students  
GROUP BY marks;
```

55. Retrieves unique marks from the students table.

```
Sql>SELECT DISTINCT marks FROM students;
```

56. Finds all students whose first name starts with the letter 'A'.

```
Sql>SELECT * FROM students  
WHERE std_name LIKE 'A%';
```

57. Retrieves students whose marks are above the average marks.

```
Sql>SELECT * FROM student  
WHERE marks > (SELECT AVG(marks) FROM student);
```